

STAT 4710/5710: Homework 4

Ying Jin

Due: November 11, 2025 at 11:59am

Contents

Instructions	1
Case study: predicting breast cancer survival with tree-based methods	3
1 Exploratory data analysis (13 points)	3
1.1 Class distribution (3 points)	3
1.2 Exploring feature distributions (10 points)	3
2 Classification trees (28 points)	4
2.1 Growing a default classification tree (12 points)	4
2.2 Tree pruning and complexity (16 points)	4
2.2.1 Fitting a large tree (10 points)	4
2.2.2 Tree pruning and cross validation (6 points)	5
3 Random forests (30 points)	6
3.1 Running a random forest with default parameters (9 points)	6
3.2 Tuning the random forests (11 points)	6
3.3 Variable importance (10 points)	6
4 Boosting (23 points)	7
4.1 Model tuning (14 points)	7
4.2 Model interpretation (9 points)	7
5 Test set evaluation and comparison (6 points)	8

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

“Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others’ solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty.”

In accordance with this policy,

Please disclose all classmates with whom you collaborated:

Please disclose which AI tools you used, and how you used them:

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use `homework-4-template.Rmd` as a starting point for your writeup, adding your solutions after `***Solution***`. Add your R code using code chunks and add your text answers using `**bold text**`. Consult the [preparing reports guide](#) for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to “print a tibble”, you should print the tibble directly.

Programming

We encourage you to follow the `tidyverse` paradigm for data visualization, manipulation, and wrangling, as in the first homework, to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you are **required** to present both code chunks and the corresponding outputs/interpretations in your PDF. In addition, it’s recommended that you:

- Use `pandas` for data manipulation (preferred, since it parallels `dplyr`).
- Use `plotnine` for visualization (preferred, since it parallels `ggplot2`). If using `seaborn/matplotlib`, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).

```
library(tidyverse)
library(cowplot)
library(glmnetUtils)
library(rpart)
library(rpart.plot)
library(kableExtra)
library(randomForest)
library(gbm)
```

Code completion. In some problems, the task is to complete an incomplete code chunk provided. In those problems, in the template, there is `eval = FALSE` in the chunk header to prevent evaluation errors. You should remove `eval = FALSE` after you have completed the code to enable evaluation and printing of results.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels:** Always include informative axis labels and, if appropriate, titles.
- **Required elements:** If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity:** Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

Case study: predicting breast cancer survival with tree-based methods

Accurate prognostic models are critically important in oncology. They can help doctors identify high-risk patients earlier, personalize treatment plans, and better allocate medical resources.

In this study, we will work with the METABRIC (Molecular Taxonomy of Breast Cancer International Consortium) dataset, which contains detailed clinical information from nearly 2,000 breast cancer patients. Each row represents a patient, and the columns include features such as:

- **age_at_diagnosis**: patient's age at the time of diagnosis
- **tumor_size**: primary tumor diameter (in mm)
- **lymph_nodes_examined_positive**: number of positive lymph nodes
- **nottingham_prognostic_index**: a standard clinical risk score
- **er_status**, **pr_status**, **her2_status**, **menopausal_status**: categorical indicators of hormone receptor and menopausal status
- **overall_survival**: status of survival outcome

We will use a binary target variable **overall_survival**, indicating whether each patient survived at least 5 years (60 months) after diagnosis. The goal is to predict this outcome using the other features.

Medical data is often noisy, complex, and different features interact in nonlinear ways. Traditional statistical methods can struggle to capture these relationships, which is why tree-based machine learning methods are especially valuable here.

Let's load the data, process categorical features, perform training-test split, and reorganize factors.

```
data = read_csv("cancer.csv")
data = data %>% mutate(overall_survival = factor(overall_survival))
data = data %>%
  mutate(across(where(is.character), as.factor))

# split data
set.seed(4710)
n <- nrow(data)
train_samples <- sample(1:n, size = 0.8 * n, replace = FALSE)
data_train <- data[train_samples,]
data_test <- data[setdiff(1:n, train_samples),]
```

1 Exploratory data analysis (13 points)

First, let's explore the training data.

1.1 Class distribution (3 points)

1. (1 points) What fraction of the tumors in the training set are survived cases?
2. (2 points) Assuming the test data contain the same class proportions, what would be the misclassification error of a naive classifier that always predicts the majority class?

1.2 Exploring feature distributions (10 points)

1. (3 points) Use a plot to compare the distributions of **tumor_size** between the survived and non-survived training samples. Use appropriate axis labels and titles.
2. (5 points) Do the same for **age_at_diagnosis** and **mutation_count**. Use appropriate axis labels, titles.
3. (2 points) What patterns did you see in the above three plots?

2 Classification trees (28 points)

From the exploratory data analysis, we can already find some features that distinguish the two groups. However, it is genuinely hard to manually identify all important features and come up with a good combination of them to predict survival. We will use classification trees to combine multiple features for better prediction.

2.1 Growing a default classification tree (12 points)

1. (2 points) Fit a classification tree with splits based on the **Gini index**, with default control parameters. Plot this tree.
2. (2 points) How many terminal nodes does this tree have? How many splits are there in the tree? [Hint: check the fitted results to get the answer.]
3. (4 point) In a terminal node, which number (from up to bottom) shows the fraction of survived samples among all the training samples belonging to that node? Which terminal node has the lowest fraction of survived samples (state the location of the terminal in the plot), and what is that fraction?
4. (4 points) What sequence of splits (specify the feature, the split point, and the direction for every split) leads to the terminal node that has the lowest fraction of survived samples?

2.2 Tree pruning and complexity (16 points)

Let's now find the optimal tree size.

2.2.1 Fitting a large tree (10 points)

1. (2 points) While we could simply prune back the default tree, there is a possibility the default tree is not large enough. In terms of the bias-variance tradeoff, why would it be a problem if the default tree were not large enough?
2. (2 points) Let us fit the deepest possible tree. This can be achieved by the function `rpart.control` with the argument `minsplit=1` (minimum number of samples required to attempt a split), `minbucket=1` (the minimum number of observations in any terminal leaf node) and `cp=0` (complexity parameter, and a value of zero leads to the deepest possible tree). Fit such a tree and plot it. (It might not be very readable, but that is okay.)
3. (1 point) Print the CP table for this tree with the following incomplete code. [Hint: the only thing you need to change is just the name of your tree fit; here we put the name `tree_large`.]

```
cp_table <- tree_large$cptable %>%
  as_tibble()
cp_table %>% kable(
  format = "latex", row.names = NA, booktabs = TRUE, digits = 5, col.names = c(
    "CP",
    "Number of splits",
    "Relative error",
    "CV error",
    "CV std"
  ),
  caption = "This is the CP table for this deepest tree."
) |>
kable_styling(position = "center") |> kable_styling(latex_options = "HOLD_position")
```

4. (5 points) How many distinct trees are there in the sequence of trees produced in part 3? How many splits does the biggest tree have? How many terminal nodes does it have? How many average observations per terminal node does it have, and why is it not 1?

2.2.2 Tree pruning and cross validation (6 points)

1. (2 points) Use the above CP table to produce the CV plot. [Hint: use the function `cv_tree(YOUR_TREE_FIT)` which is provided in the template.]
2. (2 points) Using the one-standard-error rule, how many terminal nodes does the optimal tree have? How does it compare with the default tree in Section 2.1? [Hint: use `cv_tree(YOUR_TREE_FIT)$nleaves.1se` to get the number of terminal nodes for the optimal tree using the one-standard-error rule.]
3. (2 points) Extract the optimal tree (with 1se rule) and plot it.

3 Random forests (30 points)

Note: from this point onward, your code will be somewhat time-consuming. It is recommended that you cache your code chunks using the option `cache = TRUE` in the chunk header. This way, the results of these code chunks will be saved the first time you compile them (or after you change them), making subsequent compilations much faster.

3.1 Running a random forest with default parameters (9 points)

1. (2 points) Train a random forest with default settings on `data_train`, with `overall_survival` as the response and using the function `randomForest` from the package `randomForest`. Add `set.seed(4710)` for reproducibility.
2. (3 points) Plot the out-of-bag (OOB) estimate of the misclassification error for this random forest as a function of the number of trees from 1 to 500. [Hint: first create a tibble with columns `trees` for number of trees from 1 to 500, and `oob_err` with the OOB error rate for each number of trees, which is stored in `rf_default$err.rate[, "OOB"]`. Then use `ggplot` to make the plot.]
3. (2 points) What is the OOB estimate of the misclassification error for this random forest with 200 trees?
4. (2 points) Based on OOB error plot from above, what would be a reasonable number of trees to grow without significantly compromising prediction accuracy?

3.2 Tuning the random forests (11 points)

We will now tune the random forest by varying the number of features considered at each split, `mtry`.

1. (5 points) Write your own code to train random forests with 500 trees while varying `mtry` from 5 to 30, and plot the OOB error as a function of `mtry`. **Include `set.seed(471)` at the beginning of the code chunk for reproducibility.** [Hint: for each value of `mtry`, fit a random forest with 500 trees, obtain its OOB error estimate by `YOUR_FIT$err.rate[500, 'OOB']`, and store the results in a tibble with columns `mtry` and `oob_err`. Then use `ggplot` to make the plot.]
2. (2 points) What value of `mtry` minimizes the OOB error? What is the corresponding OOB error?
3. (4 points) Using the optimal value of m , selected above, train a random forest on 500 trees just to make sure the OOB error has flattened out. Plot the OOB error of this random forest as a function of the number of trees (from 1 to 500) similar to Section 3.1, part 3, and comment on whether the error has flattened out. **Add `set.seed(471)` at the beginning for reproducibility.** Add `importance=TRUE` in the argument of `randomForest()` to make sure we can compute feature importance measures later.

3.3 Variable importance (10 points)

1. (2 points) Plot the variable importance plot for the CV-tuned optimal random forest above. Please plot the variable importance for the top 10 most important features. [Hint: use the function `varImpPlot` from the package `randomForest` using `n.var=10`.]
2. (6 points) Your feature importance plot should contain the results for two metrics: Accuracy and Gini index. What are the top-3 important features for each metric?
3. (2 points) Pick one of the features mentioned in part 2, explain whether it makes sense for it to be important in predicting breast cancer survival, including whether you would expect an increased value of this feature to indicate a greater or lesser probability of survival.

4 Boosting (23 points)

4.1 Model tuning (14 points)

As preparation for fitting boosted trees, we need to convert the response variable `overall_survival` to numeric 0-1 values.

```
data_train = data_train %>% mutate(overall_survival = 1 * (overall_survival == 1))
data_test = data_test %>% mutate(overall_survival = 1 * (overall_survival == 1))
```

1. (3 points) Fit boosted tree models with interaction depths 1, 2, and 3. For each, use a shrinkage factor of 0.1, 1000 trees, and 10-fold cross-validation. Store them into three objects. Set `n.cores = 1` and `verbose=FALSE`. **Add `set.seed(471)` at the beginning for reproducibility.**
2. (3 points) Create a tibble called `gbm_cv_results` with three columns `interaction_depth`, `ntrees`, and `cv_error`, containing the CV error for each number of trees (1:1000) and each interaction depth (1, 2, 3). [Hint: use `YOUR_BOOST_FIT$cv.error` to get the CV error for each number of trees, and use `bind_rows` to combine the results for different interaction depths.]
3. (3 points) Using the tibble above, plot the CV errors against the number of trees for each interaction depth. These three curves should be on the same plot with different colors. Also plot horizontal dashed lines at the minimal of these three curves. What are the optimal interaction depth and number of trees? [Hint: make the lineplot with `aes(col = interaction_depth)`, then add the horizontal lines by `geom_hline(yintercept = COMPUTE_MINIMUM_HERE, color = SOME_COLOR)`.]
4. (3 points) Based on the CV plot above, comment on the bias-variance tradeoff and the optimal interaction depth.
5. (2 points) Using the fitted gbm model object corresponding to the optimal interaction depth you found above, find the optimal number of trees. What is the optimal number of trees? [Hint: use function `gbm.perf(YOUR_BOOST_FIT, plot.it = FALSE)` to get the optimal number of trees.]

4.2 Model interpretation (9 points)

1. (5 points) Print the first ten rows of the relative influence table for the optimal boosting model found above (using `kable`). What are the top three features? To what extent do these align with the top three features of the random forest trained above? [Hint: use `summary(YOUR_BOOST_FIT, n.trees = optimal_num_trees, plotit = FALSE)` to get the relative influence table, and then get the top rows.]
2. (4 points) Produce partial dependence plots for the top three features based on relative influence using the incomplete code below. Comment on the nature of the relationship with the response and whether it makes sense.

```
p1 <- # TO BE COMPLETED
p2 <- # TO BE COMPLETED
p3 <- # TO BE COMPLETED
plot_grid(p1, p2, p3, align = "h")
```

5 Test set evaluation and comparison (6 points)

1. (4 points) Compute the test misclassification errors of the tuned decision tree, random forest, and boosting classifiers, collect them in a tibble and print the tibble. Which method performs best?
2. (2 points) The following code plots the misclassification errors of the random forest and boosting classifiers as a function of the number of trees, as well as a horizontal line at the misclassification error of the optimal pruned tree. The only changes needed from you is to correct the **names of fitted objects** in the code below; make them consistent with your code and run it to produce the plot. Comment on the results.

```
rf_test_err <- apply( t(apply(
  predict(rf_optimal,
    newdata = data_test,
    type = "response",
    predict.all = TRUE)$individual,
  1, function(row) (as.numeric(cummean(as.numeric(row)) > 0.5)))),
  2, function(pred) (mean(pred != data_test$overall_survival)))

gbm_test_err <- apply( predict(boost_depth3, newdata = data_test,
  type = "response", n.trees = 1:500 ), 2,
  function(p) (mean(as.numeric(p > 0.5) != data_test$overall_survival)) )

rf_test_err_tibble <- tibble(ntree = 1:500, error = rf_test_err, method = 'Random forest')
gbm_test_err_tibble <- tibble(ntree = 1:500, error = gbm_test_err, method = 'Boosting')
test_errors <- bind_rows(rf_test_err_tibble, gbm_test_err_tibble)
test_errors |> ggplot(aes(x = ntree, y = error, group = method)) +
  geom_line(aes(color = method)) +
  geom_hline(yintercept = tree_test_error,
    linetype = "dashed", color = "blue") +
  scale_y_log10() +
  scale_color_manual(
    values = c("red", "green") )+
  labs(x = "Number of trees",
    y = "Misclassification error on log scale", color = "Model") +
  theme_bw()
```